

```
[6]
✓ 29s ▶ def dfs(state, goal, visited, path, cost, limit):

    # Goal test
    if state == goal:
        return path, cost

    # Stop if depth limit reached
    if cost == limit:
        return None, None

    visited.add(state)

    # Position of blank
    blank = state.index(0)

    # Possible moves
    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }

    # Generate next states
    for new_position in moves[blank]:

        new_state = list(state)

        # Swap blank
        new_state[blank], new_state[new_position] = \
            new_state[new_position], new_state[blank]

        new_state = tuple(new_state)
```



8_puzzle.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all ▼



[6]

✓ 29%



```
        new_state[new_position], new_state[blank]

    new_state = tuple(new_state)

    if new_state not in visited:

        result_path, result_cost = dfs(
            new_state,
            goal,
            visited,
            path + [new_state],
            cost + 1,
            limit
        )

        if result_path is not None:
            return result_path, result_cost

    return None, None

# Input
print("Enter initial state:")
initial = tuple(map(int, input().split()))

print("Enter goal state:")
goal = tuple(map(int, input().split()))

# DFS depth limit
limit = 30

visited = set()

path, cost = dfs(initial, goal, visited, [initial], 0, limit)

# Output
if path:
    print("\nSolution found!")
    print("Path Cost:", cost)

    print("\nPath:")
```



8_puzzle.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[6]

✓ 29s



```
print("\nSolution found!")
print("Path Cost:", cost)

print("\nPath:")
for state in path:
    print(state)

else:
    print("No solution found.")

print(["Deeksha R Naik (1WN24CS086)"])
```

```
... Enter initial state:
1 2 3 0 4 6 7 5 8
Enter goal state:
1 2 3 4 5 6 7 8 0
```

```
Solution found!
Path Cost: 13
```

```
Path:
(1, 2, 3, 0, 4, 6, 7, 5, 8)
(0, 2, 3, 1, 4, 6, 7, 5, 8)
(2, 0, 3, 1, 4, 6, 7, 5, 8)
(2, 4, 3, 1, 0, 6, 7, 5, 8)
(2, 4, 3, 0, 1, 6, 7, 5, 8)
(0, 4, 3, 2, 1, 6, 7, 5, 8)
(4, 0, 3, 2, 1, 6, 7, 5, 8)
(4, 1, 3, 2, 0, 6, 7, 5, 8)
(4, 1, 3, 0, 2, 6, 7, 5, 8)
(0, 1, 3, 4, 2, 6, 7, 5, 8)
(1, 0, 3, 4, 2, 6, 7, 5, 8)
(1, 2, 3, 4, 0, 6, 7, 5, 8)
(1, 2, 3, 4, 5, 6, 7, 0, 8)
(1, 2, 3, 4, 5, 6, 7, 8, 0)
Deeksha R Naik (1WN24CS086)
```



8_puzzle.ipynb



File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all



✓ 21s

print(DEKSH R NAIK (1W1Z4C3000))

```
... Enter initial state:
5 4 0 6 1 8 7 3 2
Enter goal state:
0 1 2 3 4 5 6 7 8
```

```
Solution found!
Path Cost: 48
```

```
Path:
(5, 4, 0, 6, 1, 8, 7, 3, 2)
(5, 0, 4, 6, 1, 8, 7, 3, 2)
(0, 5, 4, 6, 1, 8, 7, 3, 2)
(6, 5, 4, 0, 1, 8, 7, 3, 2)
(6, 5, 4, 1, 0, 8, 7, 3, 2)
(6, 0, 4, 1, 5, 8, 7, 3, 2)
(0, 6, 4, 1, 5, 8, 7, 3, 2)
(1, 6, 4, 0, 5, 8, 7, 3, 2)
(1, 6, 4, 5, 0, 8, 7, 3, 2)
(1, 0, 4, 5, 6, 8, 7, 3, 2)
(0, 1, 4, 5, 6, 8, 7, 3, 2)
(5, 1, 4, 0, 6, 8, 7, 3, 2)
(5, 1, 4, 6, 0, 8, 7, 3, 2)
(5, 1, 4, 6, 8, 0, 7, 3, 2)
(5, 1, 0, 6, 8, 4, 7, 3, 2)
(5, 0, 1, 6, 8, 4, 7, 3, 2)
(0, 5, 1, 6, 8, 4, 7, 3, 2)
(6, 5, 1, 0, 8, 4, 7, 3, 2)
(6, 5, 1, 8, 0, 4, 7, 3, 2)
(6, 0, 1, 8, 5, 4, 7, 3, 2)
(0, 6, 1, 8, 5, 4, 7, 3, 2)
(8, 6, 1, 0, 5, 4, 7, 3, 2)
(8, 6, 1, 5, 0, 4, 7, 3, 2)
(8, 6, 1, 5, 3, 4, 7, 0, 2)
(8, 6, 1, 5, 3, 4, 0, 7, 2)
(8, 6, 1, 0, 3, 4, 5, 7, 2)
(8, 6, 1, 3, 0, 4, 5, 7, 2)
(8, 6, 1, 3, 4, 0, 5, 7, 2)
(8, 6, 1, 3, 4, 2, 5, 7, 0)
(8, 6, 1, 3, 4, 2, 5, 0, 7)
(8, 6, 1, 3, 0, 2, 5, 4, 7)
(8, 0, 1, 3, 6, 2, 5, 4, 7)
(0, 8, 1, 3, 6, 2, 5, 4, 7)
(3, 8, 1, 0, 6, 2, 5, 4, 7)
```



8_puzzle.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



```
(1, 6, 4, 5, 0, 8, 7, 3, 2)
(1, 0, 4, 5, 6, 8, 7, 3, 2)
...
(0, 1, 4, 5, 6, 8, 7, 3, 2)
(5, 1, 4, 0, 6, 8, 7, 3, 2)
(5, 1, 4, 6, 0, 8, 7, 3, 2)
(5, 1, 4, 6, 8, 0, 7, 3, 2)
(5, 1, 0, 6, 8, 4, 7, 3, 2)
(5, 0, 1, 6, 8, 4, 7, 3, 2)
(0, 5, 1, 6, 8, 4, 7, 3, 2)
(6, 5, 1, 0, 8, 4, 7, 3, 2)
(6, 5, 1, 8, 0, 4, 7, 3, 2)
(6, 0, 1, 8, 5, 4, 7, 3, 2)
(0, 6, 1, 8, 5, 4, 7, 3, 2)
(8, 6, 1, 0, 5, 4, 7, 3, 2)
(8, 6, 1, 5, 0, 4, 7, 3, 2)
(8, 6, 1, 5, 3, 4, 7, 0, 2)
(8, 6, 1, 5, 3, 4, 0, 7, 2)
(8, 6, 1, 0, 3, 4, 5, 7, 2)
(8, 6, 1, 3, 0, 4, 5, 7, 2)
(8, 6, 1, 3, 4, 0, 5, 7, 2)
(8, 6, 1, 3, 4, 2, 5, 7, 0)
(8, 6, 1, 3, 4, 2, 5, 0, 7)
(8, 6, 1, 3, 0, 2, 5, 4, 7)
(8, 0, 1, 3, 6, 2, 5, 4, 7)
(0, 8, 1, 3, 6, 2, 5, 4, 7)
(3, 8, 1, 0, 6, 2, 5, 4, 7)
(3, 8, 1, 5, 6, 2, 0, 4, 7)
(3, 8, 1, 5, 6, 2, 4, 0, 7)
(3, 8, 1, 5, 0, 2, 4, 6, 7)
(3, 0, 1, 5, 8, 2, 4, 6, 7)
(3, 1, 0, 5, 8, 2, 4, 6, 7)
(3, 1, 2, 5, 8, 0, 4, 6, 7)
(3, 1, 2, 5, 0, 8, 4, 6, 7)
(3, 1, 2, 0, 5, 8, 4, 6, 7)
(3, 1, 2, 4, 5, 8, 0, 6, 7)
(3, 1, 2, 4, 5, 8, 6, 0, 7)
(3, 1, 2, 4, 5, 8, 6, 7, 0)
(3, 1, 2, 4, 5, 0, 6, 7, 8)
(3, 1, 2, 4, 0, 5, 6, 7, 8)
(3, 1, 2, 0, 4, 5, 6, 7, 8)
(0, 1, 2, 3, 4, 5, 6, 7, 8)
```

Deeksha R Naik (1WN24CS086)